

MODUL PRAKTIKUM

PROJECT APLIKASI WEB

Pertemuan 3

CSS BUKAN SEKADAR WARNA

Membuat Tampilan yang Teratur, Konsisten, dan Bisa Dijelaskan

“HTML memberi halaman struktur. CSS memberi struktur itu ritme visual. Tugas kita bukan membuatnya ramai, tetapi membuat informasi lebih mudah dipahami.”

Program Studi	Teknologi Informasi
Semester	1
Durasi Praktikum	Rekomendasi 170 menit
Karakter Pertemuan	Discovery CSS + guided styling + DevTools + debugging

Cara Menggunakan Modul Ini

Pertemuan 3 melanjutkan project TechStarter Day dari Pertemuan 2. Struktur HTML Anda sudah memiliki makna. Hari ini Anda tidak diminta membuat desain yang “wah”, tetapi belajar membuat tampilan yang teratur, konsisten, mudah dibaca, dan dapat dijelaskan berdasarkan aturan CSS. Setiap kali sebuah style tidak bekerja, jangan langsung menambah !important. Cari selector yang aktif, aturan yang menang, dan bukti di DevTools.

ATURAN PRAKTIK

Ketik sendiri CSS inti. Jangan mengubah banyak properti sekaligus saat terjadi masalah. Gunakan pola GEJALA → PREDIKSI → BUKTI DI DEVTOOLS → SATU PERUBAHAN → VERIFIKASI. Jangan memakai template CSS, framework, atau AI untuk menghasilkan stylesheet utama.

Identitas Praktikan

Isian	Jawaban
Nama	
NIM	
Squad	
Role Pertemuan 3	
Laptop / PC	
Browser	

Capaian Pertemuan 3

Setelah menyelesaikan praktikum, mahasiswa diharapkan mampu:

1. Menjelaskan hubungan HTML dan CSS dengan bahasa sendiri serta membedakan struktur konten dari presentasi visual.
2. Menulis aturan CSS yang terdiri dari selector, property, dan value serta memilih selector yang sesuai dengan tujuan.
3. Menjelaskan secara sederhana bagaimana cascade, specificity, dan source order menentukan aturan yang digunakan browser.
4. Menggunakan class untuk style yang dapat dipakai ulang dan menghindari ketergantungan pada id untuk styling umum.
5. Menerapkan prinsip visual hierarchy melalui typography, warna, jarak, dan konsistensi.
6. Menjelaskan box model: content, padding, border, margin, serta fungsi box-sizing: border-box.
7. Menggunakan DevTools Styles dan Computed untuk memeriksa aturan aktif, aturan yang kalah, serta ukuran box.
8. Mendiagnosis CSS yang tidak termuat atau tidak berlaku menggunakan bukti, bukan tebakan.
9. Menyimpan checkpoint project dengan commit Git yang menjelaskan perubahan visual yang dibuat.

Preflight : Bawa Project Pertemuan 2

- ☐ Folder P02_HTML-Semantic dapat dibuka di VS Code.
- ☐ index.html memiliki header, nav, main, section, dan footer yang dapat dijelaskan.
- ☐ Link navigasi internal bekerja.
- ☐ Gambar pada folder assets/images dapat dimuat dengan path relatif.
- ☐ Live Server dan DevTools dapat digunakan.
- ☐ git status berjalan dan project memiliki commit Pertemuan 2.

JIKA PROJECT P02 BELUM RAPI

Jangan mulai dengan menghapus semuanya. Gunakan versi terakhir yang masih bekerja. Dosen hanya membantu memulihkan baseline minimum. Pertemuan 3 harus fokus pada CSS dan cara membuktikan style yang berlaku.

Mission Brief : Halaman Sudah Benar, Tetapi Sulit Dibaca

TechStarter Day akan dipublikasikan kepada mahasiswa baru. Secara struktur, halaman dari Pertemuan 2 sudah cukup baik. Masalahnya: semua elemen terlihat hampir sama penting, jarak antarbagian tidak konsisten, teks terlalu melebar, dan link navigasi tidak terasa sebagai bagian dari interface.

PERINTAH ATASAN

“Rapihan halaman tanpa mengubah makna HTML. Buat informasi penting cepat ditemukan, gaya antarbagian konsisten, dan pastikan setiap keputusan visual dapat kalian jelaskan. Jangan menambah library atau framework.”

Percobaan Pertama : Ketika Tiga Aturan Berebut Satu Elemen

Sebelum mempercantik project utama, kita akan membuat laboratorium CSS kecil. Buat folder P03_CSS-Lab. Di dalamnya buat index.html dan style.css berikut.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Lab</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="container">
    <h1>CSS Lab</h1>
    <p id="status" class="info">Saya sedang berebut warna.</p>
  </main>
</body>
</html>
```

style.css

```
p {
  color: green;
}

.info {
  color: blue;
}

#status {
  color: red;
}
```

1. Jalankan dengan Live Server. Jangan mengubah kode lebih dulu.
2. Prediksi warna teks sebelum melihat browser. Catat prediksi Anda.
3. Buka DevTools > Elements, klik paragraf, lalu lihat panel Styles.
4. Cari rule yang dicoret. Rule yang dicoret bukan berarti tidak terbaca; browser hanya memilih rule lain yang lebih kuat.
5. Hapus sementara blok #status, simpan, dan lihat siapa yang menang berikutnya.
6. Pindahkan rule p ke bagian paling bawah. Apakah p sekarang mengalahkan .info? Jangan menebak: buktikan lewat Styles.

Pertanyaan Pemandu

1. Mengapa browser tidak menggabungkan tiga nilai color menjadi satu warna baru?

2. Apa perbedaan selector elemen p, selector class .info, dan selector id #status?

3. Apakah rule yang ditulis paling bawah selalu menang? Dalam kondisi apa source order baru menjadi penentu?

4. Jika sebuah rule dicoret di DevTools, apakah file CSS gagal dibaca? Apa buktinya?

5. Mengapa memakai id untuk semua styling dapat membuat stylesheet lebih sulit dikendalikan nanti?

Konsep Baru yang Ditemukan : CSS, Cascade, dan Specificity

CSS adalah kumpulan aturan presentasi. Satu aturan biasanya memiliki selector yang memilih elemen, lalu declaration yang berisi pasangan property dan value. Ketika beberapa aturan memengaruhi property yang sama, browser menjalankan proses cascade untuk menentukan nilai yang dipakai.

Istilah	Makna sederhana	Contoh
Selector	Menentukan elemen mana yang menjadi target	.card, nav a, h2
Property	Aspek tampilan yang ingin diubah	color, padding, border
Value	Nilai yang diberikan pada property	1rem, #1f3f68, solid
Cascade	Proses browser memilih aturan yang berlaku	Beberapa rule memengaruhi color
Specificity	Kekuatan relatif selector	id biasanya lebih spesifik daripada class
Source order	Urutan penulisan sebagai penentu saat kekuatan setara	Rule setara yang ditulis belakangan dapat menang
Inheritance	Beberapa property diwariskan dari parent	color dan font-family sering diwariskan

MENTAL MODEL

Jangan hafal angka specificity dulu. Untuk tahap ini gunakan urutan berpikir: “Apakah property yang sama datang dari beberapa rule? Selector mana yang lebih spesifik? Jika setara, mana yang datang belakangan?” Setelah itu cek DevTools.

Bagian A : Siapkan CSS Utama Project TechStarter Day

A1. Salin Baseline dengan Aman

Buat folder baru P03_CSS-Foundation. Salin index.html dan folder assets dari Pertemuan 2. Buat style.css baru agar perubahan hari ini dapat dibandingkan dengan baseline sebelumnya.

```
P03_CSS-Foundation/
├── index.html
├── style.css
├── assets/
│   └── images/
│       └── lab-komputer.jpg
```

Pastikan head pada index.html memiliki link berikut:

```
<link rel="stylesheet" href="style.css">
```

CHECKPOINT A1

Buka Network lalu refresh. Cari style.css dan pastikan request berhasil. Jika halaman sama sekali tidak berubah setelah CSS ditulis, verifikasi dulu apakah file benar benar termuat sebelum mengutak atik selector.

A2. Mulai dari Aturan Global yang Masuk Akal

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #1f2937;
  background: #f5f7fa;
  line-height: 1.6;
}

img {
  max-width: 100%;
  height: auto;
}
```

Jangan sekadar menerima kode tersebut. Setelah mengetik, uji satu per satu: nonaktifkan color, background, line-height, atau max-width melalui DevTools. Catat properti mana yang paling terasa pengaruhnya terhadap keterbacaan.

A3. Kenali Selector yang Akan Dipakai Hari Ini

Selector	Contoh	Kapan dipakai
Elemen	h1, p, img	Saat semua elemen sejenis memang berbagi aturan dasar
Class	.container, .card	Untuk pola visual yang dapat digunakan berulang
Descendant	nav a	Untuk elemen a yang berada di dalam nav
Pseudo class	a:hover	Untuk keadaan tertentu tanpa menambah elemen HTML
ID	#agenda	Utamanya untuk identitas unik / anchor; jangan dijadikan kebiasaan styling umum

ATURAN PRAKTIS

Jika gaya visual dapat dipakai ulang pada beberapa elemen, class biasanya lebih mudah dirawat daripada id. HTML menjawab “ini konten apa”, class CSS dapat membantu menjawab “pola visual apa yang dipakai”.

Bagian B : Visual Hierarchy Sebelum Dekorasi

Visual hierarchy adalah cara tampilan membantu mata memahami urutan kepentingan. Anda tidak perlu memakai banyak warna. Mulailah dari lebar konten, typography, dan jarak yang konsisten.

B1. Batasi Lebar Konten

Tambahkan class="container" pada elemen utama yang membungkus konten, atau gunakan selector main bila struktur Anda sederhana. Gunakan salah satu pendekatan secara konsisten.

```
main {
  width: min(92%, 900px);
  margin: 0 auto;
  padding: 2rem 0 3rem;
}
```

Ubah 900px menjadi 1400px lalu 600px dan baca ulang paragraf. Lebar yang “lebih besar” tidak selalu lebih nyaman dibaca. Tujuan max-width adalah mengendalikan panjang baris, bukan sekadar mengecilkan halaman.

B2. Bentuk Hierarchy Typography

```
h1,
h2,
h3 {
  color: #17365d;
  line-height: 1.2;
}

h1 {
  font-size: 2rem;
  margin-bottom: 0.5rem;
}

h2 {
  margin-top: 2rem;
  margin-bottom: 0.75rem;
}

p {
  margin-top: 0;
}
```

Lihat halaman tanpa menilai warna terlebih dahulu. Tanyakan: apakah h1 jelas sebagai pintu masuk halaman? Apakah h2 membantu scanning? Apakah jarak sebelum section cukup membedakan satu kelompok informasi dari kelompok berikutnya?

B3. Style Header, Navigasi, dan Footer

```
header {
  background: #17365d;
  color: white;
  padding: 2rem 4%;
}

header h1 {
  color: white;
}

nav {
  background: white;
  padding: 0.75rem 4%;
  border-bottom: 1px solid #dbe2ea;
}

nav a {
  color: #17365d;
  font-weight: 700;
  text-decoration: none;
  margin-right: 1rem;
}

nav a:hover {
  text-decoration: underline;
}

footer {
  background: #e9eef5;
  padding: 1.25rem 4%;
}
```

WHY : HOVER BUKAN DEKORASI

Perubahan saat pointer berada di atas link dapat memberi feedback bahwa elemen tersebut interaktif. Tetapi link tidak boleh bergantung pada hover saja untuk diketahui sebagai link. Teks, posisi, atau gaya dasar tetap harus memberi petunjuk.

Bagian C : Box Model — Mengapa “Lebarnya 300px” Bisa Lebih Besar?

Setiap elemen dapat dibayangkan sebagai kotak yang memiliki empat lapisan: content, padding, border, dan margin. Banyak kebingungan layout berasal dari tidak membedakan ruang di dalam box dengan ruang antarbox.

Lapisan	Letak	Pertanyaan sederhana
Content	Isi terdalam	Berapa area untuk teks atau gambar?
Padding	Di dalam border	Seberapa jauh isi dari tepi kotak?
Border	Garis pembatas box	Di mana batas visual elemen?
Margin	Di luar border	Seberapa jauh box ini dari box lain?

C1. Buat Card Sederhana

Tambahkan class="card" pada article yang mewakili satu blok informasi. Jika belum ada article yang sesuai, gunakan pada section tertentu yang memang ingin diperlakukan sebagai satu panel visual.

```
.card {
  background: white;
  border: 1px solid #dbe2ea;
  border-radius: 10px;
  padding: 1.25rem;
  margin-bottom: 1rem;
}
```

Buka DevTools > Elements, pilih `.card`, lalu lihat diagram box model pada panel Computed. Arahkan pointer ke padding dan margin. Browser akan menyorot area yang berbeda pada halaman.

CHECKPOINT BOX MODEL

Tunjukkan kepada teman satu squad: mana content, padding, border, dan margin pada elemen card Anda. Jika hanya bisa menyebutkan definisi tanpa menunjuk bukti visual, checkpoint belum selesai.

C2. Eksperimen box-sizing

Di CSS Lab, buat sebuah kotak berikut lalu amati ukurannya di DevTools.

```
.kotak-uji {
  width: 300px;
  padding: 30px;
  border: 10px solid #17365d;
  background: #d9eaf7;
}
```

1. Matikan rule `* { box-sizing: border-box; }` jika masih aktif.
2. Catat lebar total box yang terlihat di Computed.
3. Aktifkan kembali `box-sizing: border-box`.
4. Bandingkan lebar totalnya.
5. Jelaskan dengan kata sendiri mengapa `box-sizing: border-box` membantu ketika kita menentukan width.

Bagian D : Cascade Lab di DevTools

Sekarang kembali ke halaman TechStarter Day. Tujuan bagian ini bukan menambah banyak CSS, tetapi belajar membaca alasan ketika suatu aturan tidak berlaku.

D1. Buat Konflik yang Sengaja

```
section p {
  color: #374151;
}

.highlight {
  color: #0b6b3a;
}

#tentang p {
  color: #8b1e3f;
}
```

Tambahkan `class="highlight"` pada satu paragraf di `section id="tentang"`. Sebelum refresh, prediksi warna. Setelah refresh, pilih paragraf tersebut di Elements lalu lihat Styles. Catat rule mana yang menang dan rule mana yang dicoret.

Prediksi saya:

Bukti dari DevTools:

Kesimpulan tentang selector yang menang:

D2. Source Order Hanya Berlaku Saat “Kekuatan” Setara

```
.note {
  border-left: 4px solid #17365d;
}

.note {
  border-left-color: #c07a00;
}
```

Karena kedua selector sama, rule yang datang belakangan dapat mengganti property yang sama. Pindahkan urutannya dan verifikasi. Ini contoh source order; jangan menyimpulkan bahwa “yang paling bawah selalu menang” untuk semua kasus.

Bagian E : Break It on Purpose

Lakukan error satu per satu. Setelah setiap error, kembalikan project ke kondisi benar. Catat gejala, bukti, satu perubahan, dan hasil verifikasi.

E1. CSS Tidak Termuat Sama Sekali

1. Ubah href="style.css" menjadi href="styles.css" sementara file yang benar tetap bernama style.css.
2. Refresh halaman dan amati perubahan besar pada seluruh tampilan.
3. Buka Network, refresh lagi, dan cari request CSS yang gagal.
4. Bandingkan nama pada href dengan nama file di Explorer.
5. Perbaiki satu karakter / nama file, refresh, lalu verifikasi style kembali.

Gejala:

Bukti Network:

Perbaikan tunggal:

Hasil verifikasi:

E2. CSS Termuat, Tetapi Rule Anda “Kalah”

1. Pastikan style.css termuat normal.
2. Buat .highlight { color: blue; } untuk paragraf di dalam #tentang.
3. Tambahkan #tentang p { color: red; } setelah atau sebelum rule tersebut.
4. Di Styles, cari deklarasi color yang dicoret.

5. Jangan memakai !important. Tentukan apakah konflik benar benar perlu ada. Jika class harus mengendalikan tampilan, sederhanakan selector yang terlalu spesifik atau ubah struktur aturan.

Rule yang saya harapkan menang:

Rule yang benar benar menang:

Bukti Styles / Computed:

Perubahan paling sederhana yang saya pilih:

LARANGAN HARI INI

Jangan memakai !important sebagai jalan pintas. !important bukan “obat CSS error”. Pada tahap ini, konflik harus diselesaikan dengan memahami selector, urutan, dan kebutuhan desain.

Bagian F : Build Final Versi Pertemuan 3

Rapikan kembali stylesheet. Hapus rule eksperimen yang tidak dibutuhkan. Halaman final tidak harus sama persis antar mahasiswa, tetapi keputusan visual harus konsisten dan dapat dijelaskan.

Checklist Minimum Tampilan

- ☐ style.css termuat tanpa request gagal.
- ☐ body memiliki typography dan line-height yang nyaman dibaca.
- ☐ lebar konten utama dibatasi sehingga paragraf tidak terlalu panjang.
- ☐ h1 dan h2 memiliki hierarchy yang jelas tanpa mengubah struktur heading HTML.
- ☐ header, nav, main, dan footer mudah dibedakan secara visual.
- ☐ minimal satu class reusable digunakan untuk pola visual seperti card, note, atau badge.
- ☐ minimal satu elemen menunjukkan penggunaan padding, border, dan margin yang dapat dijelaskan.
- ☐ gambar tidak meluber dari container.
- ☐ link navigasi memiliki keadaan dasar yang jelas dan feedback hover sederhana.
- ☐ tidak ada penggunaan !important.
- ☐ tidak ada selector id yang dipakai sebagai pola styling berulang.
- ☐ DevTools Styles dan Computed dapat digunakan untuk menjelaskan satu keputusan CSS.

Bagian G : Style Audit dengan Teman Satu Squad

Bertukar laptop dengan satu teman. Reviewer tidak boleh memperbaiki kode. Tugas reviewer adalah menemukan satu kekuatan dan satu masalah berdasarkan bukti.

Item Review	Ya / Belum	Catatan reviewer
Hierarchy judul mudah dipahami		
Panjang baris paragraf nyaman dibaca		
Jarak antarbagian konsisten		
Class digunakan untuk pola reusable		
Padding dan margin tidak tertukar fungsinya		
Tidak ada !important		

Item Review	Ya / Belum	Catatan reviewer
Rule yang kalah dapat ditemukan di Styles		
Box model dapat ditunjukkan di Computed		

Bagian H : Git Checkpoint Kedua

Sebelum commit, bersihkan eksperimen yang tidak dipakai. Lihat perubahan dengan status, lalu simpan checkpoint.

```
git status
git add .
git commit -m "feat: terapkan fondasi CSS dan visual hierarchy"
git log --oneline --max-count=3
```

JANGAN COMMIT FILE ACAK

Pastikan folder project tidak berisi screenshot besar, file installer, atau hasil download yang tidak relevan. Commit seharusnya merekam source yang memang diperlukan project.

Output yang Harus Dikumpulkan

Kode	Output	Jenis	Bukti minimum
P3.1	Folder P03_CSS-Foundation	Individu	index.html, style.css, assets
P3.2	Cascade Lab	Individu	Catatan prediksi + screenshot Styles
P3.3	Box Model Experiment	Individu	Perbandingan sebelum / sesudah border-box
P3.4	Break It : CSS path	Individu	Gejala, Network evidence, verifikasi
P3.5	Break It : specificity	Individu	Rule menang / kalah + alasan
P3.6	Final styled page	Individu	Halaman konsisten dan dapat dijelaskan
P3.7	Style Audit	Squad	Tabel review silang
P3.8	Git checkpoint	Individu	git log oneline + commit relevan
P3.9	Post test + refleksi	Individu	Jawaban tertulis

NAMA FOLDER PENGUMPULAN

P03_NIM_NamaSingkat. Simpan source code, bukti debugging, dan satu screenshot tampilan final. Jangan mengumpulkan node_modules atau installer apa pun.

Post Test Pertemuan 3

1. Apa perbedaan selector, property, dan value pada CSS?

2. Jika .info dan #status sama sama mengatur color, mengapa salah satunya dapat menang?

3. Kapan source order menjadi penentu?

4. Mengapa class umumnya lebih cocok daripada id untuk pola visual yang ingin dipakai ulang?

5. Jelaskan perbedaan padding dan margin menggunakan contoh sebuah card.

6. Apa fungsi box-sizing: border-box?

7. Jika style.css tidak memberi efek sama sekali, bukti pertama apa yang Anda cari?

8. Jika sebuah declaration dicoret di panel Styles, apa artinya?

9. Mengapa !important tidak boleh dijadikan solusi pertama?

10. Sebutkan dua keputusan visual pada halaman Anda dan WHY di baliknya.

Refleksi Individu

1. Properti CSS yang sebelumnya saya anggap mudah tetapi ternyata memiliki efek samping adalah:

2. Bukti DevTools yang paling membantu saya hari ini adalah:

3. Saya paling sering tertukar antara padding dan margin ketika:

4. Satu keputusan styling yang saya ubah setelah melihat halaman sebagai pengguna adalah:

5. Jika saya harus menjelaskan cascade kepada teman yang belum pernah coding, saya akan mengatakan:

6. Satu hal yang ingin saya rapikan lagi sebelum Pertemuan 4 adalah:

Checklist Sebelum Meninggalkan Laboratorium

- ☐ Halaman final masih memiliki semantic HTML dari Pertemuan 2.
- ☐ style.css termuat dan tidak ada error resource utama.
- ☐ Semua rule eksperimen yang tidak perlu sudah dibersihkan.
- ☐ Saya dapat menunjukkan box model satu elemen di Computed.
- ☐ Saya dapat menunjukkan satu declaration yang kalah / menang di Styles.
- ☐ Git commit Pertemuan 3 berhasil.
- ☐ Folder pengumpulan mengikuti nama yang diminta.
- ☐ Saya mampu menjelaskan WHY dari minimal dua keputusan visual tanpa membaca kode.

PREVIEW PERTEMUAN 4

Project sekarang sudah mempunyai struktur dan tampilan. Pertanyaan berikutnya: bagaimana source code ini disimpan, dibagikan, dilacak perubahannya, dan dikolaborasikan tanpa saling menimpa file? Pertemuan 4 akan membawa Git dari sekadar “commit lokal” menuju workflow repository dan GitHub.

Sumber Belajar Lanjutan

MDN Web Docs : CSS first steps : https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Getting_started

MDN Web Docs : Cascade and inheritance : https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Handling_conflicts

MDN Web Docs : CSS box model : https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Box_model

Catatan: gunakan referensi untuk memahami konsep, bukan menyalin satu stylesheet utuh. Project harus tetap dapat Anda jelaskan baris demi baris pada level konsep.